SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 1 de 4 · Cómo un modelo genera texto y cuánta memoria exige

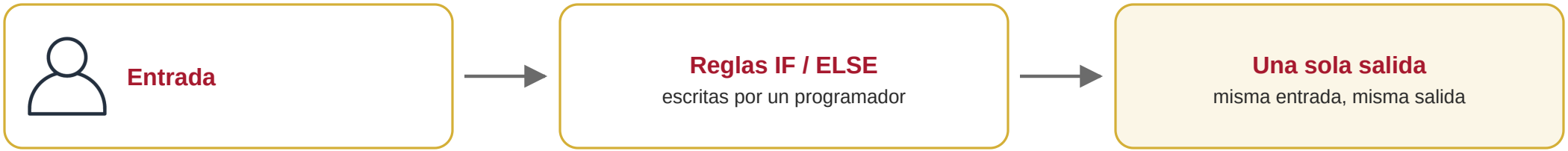
Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 1 de 4 · Cómo un modelo genera texto y cuánta memoria exige
IA = inteligencia artificial

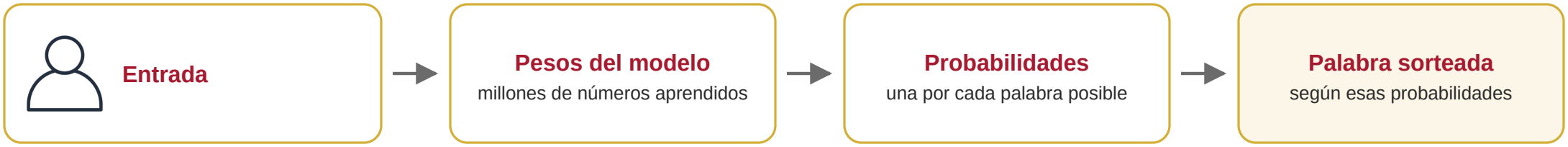
Del software determinista al probabilístico

Reglas escritas a mano frente a probabilidades aprendidas

Código tradicional



Modelo de lenguaje (LLM)



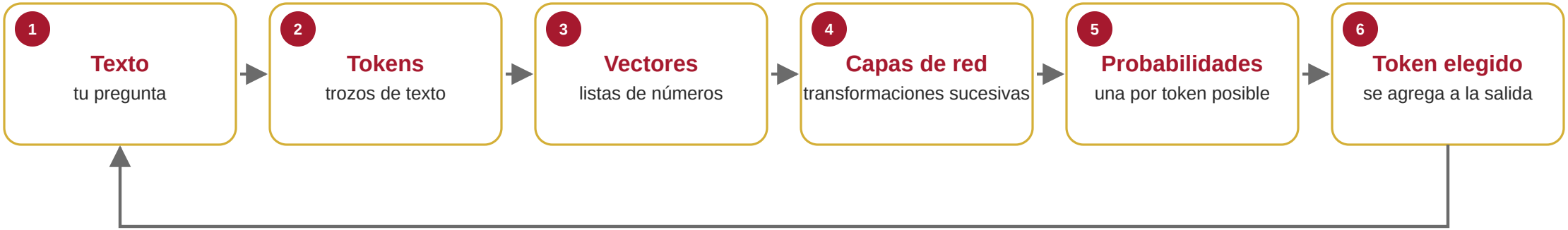
Misma pregunta, respuestas que pueden variar.

IF = condición «si» · ELSE = alternativa «si no»

Fuente: AWS Well-Architected, Generative AI Lens (docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens)

Un modelo de lenguaje por dentro

Del texto de entrada al token de salida



Archivo de pesos
más un programa en la GPU

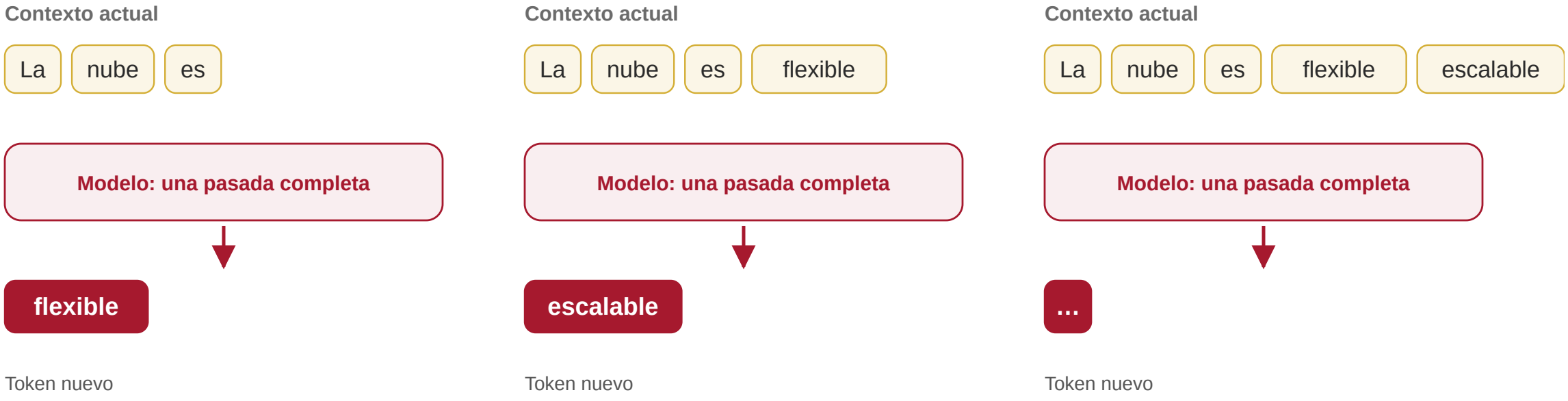
API gestionada
o servidor propio con GPU

GPU = unidad de procesamiento gráfico · API = interfaz de programación

Fuente: Amazon Bedrock User Guide, Models at a glance (docs.aws.amazon.com/bedrock/latest/userguide/model-cards.html)

Un token a la vez

El modelo no escribe la respuesta de golpe



Se agrega al contexto y el ciclo continúa.

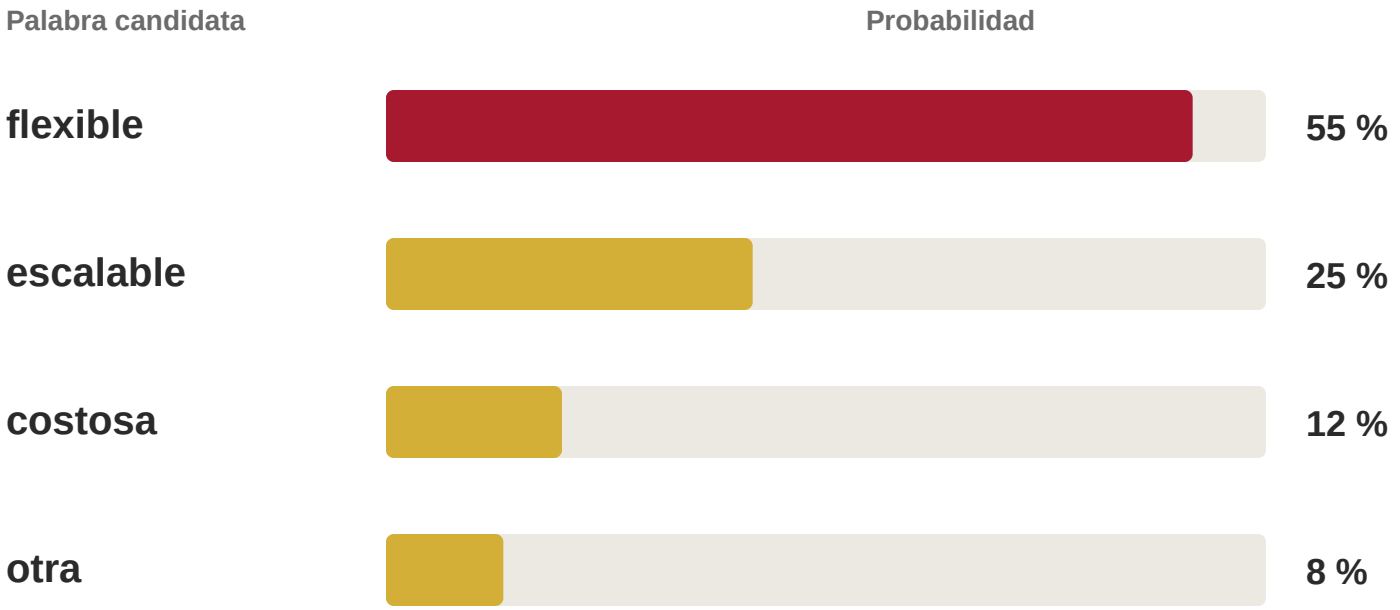
Respuesta larga: muchas pasadas.

Ejemplo ilustrativo de generación de texto.

Fuente: Amazon Bedrock User Guide, Inference parameters (docs.aws.amazon.com/bedrock/latest/userguide/inference-parameters.html)

Cada paso es una apuesta ponderada

El modelo reparte 100 % entre las palabras posibles



Suma: 100 %

Sorteo

flexible

escalable

costosa

otra

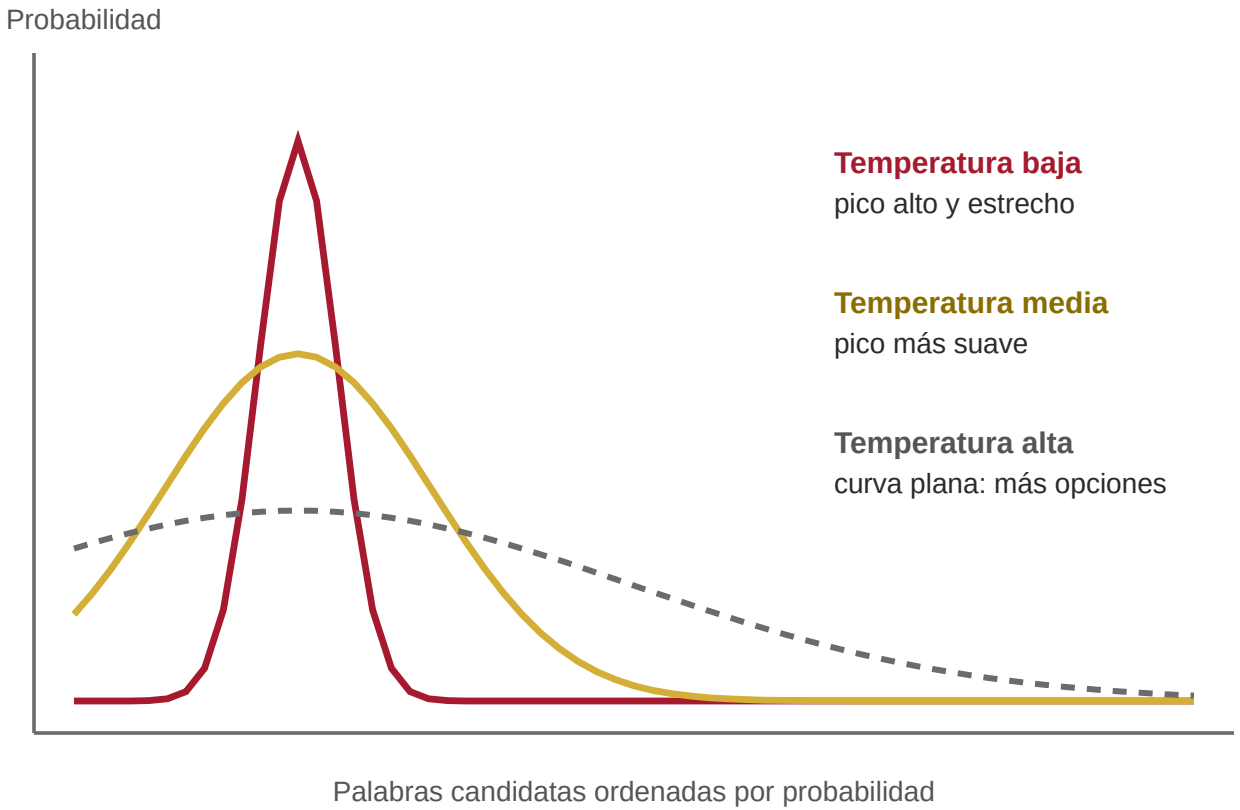
La palabra más probable sale más veces, pero no siempre.

Valores ilustrativos: no provienen de un modelo concreto.

Fuente: Amazon Bedrock User Guide, Inference parameters (consulta 2026-11-05); valores ilustrativos

Temperatura: afila o aplana la curva

Un solo parámetro cambia el carácter de la respuesta



Temperatura baja
pico alto y estrecho

Temperatura media
pico más suave

Temperatura alta
curva plana: más opciones

Temperatura baja
SQL: estable y repetible

Temperatura media
chat de atención al cliente: natural

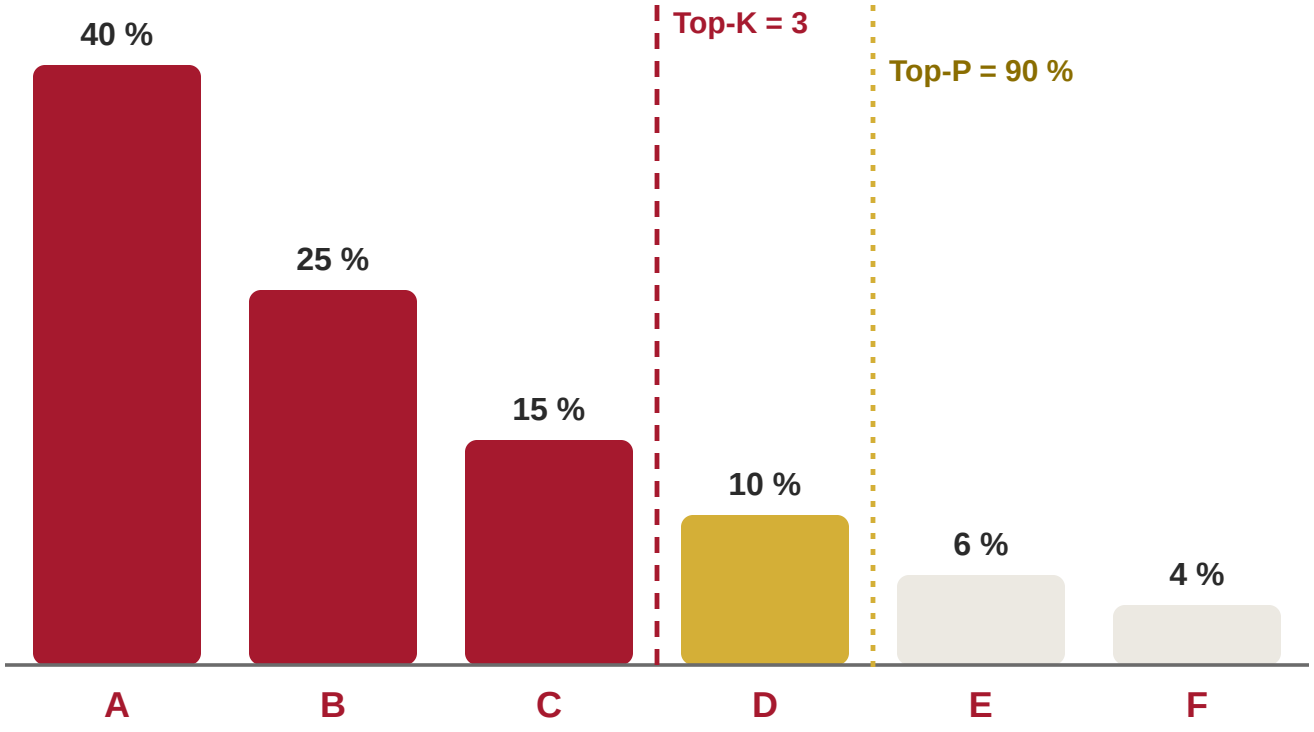
Temperatura alta
lluvia de ideas: creativa

SQL = lenguaje de consulta estructurado

Fuente: Amazon Bedrock User Guide, Inference parameters (docs.aws.amazon.com/bedrock/latest/userguide/inference-parameters.html)

Top-K y Top-P: dónde cortar la lista

Dos formas de descartar candidatos improbables



Top-K corta por cantidad

A, B, C

Top-P corta por probabilidad

40 + 25 + 15 + 10 = 90 %

Las palabras fuera del corte nunca se sortean.

Fuente: Amazon Bedrock User Guide, Inference parameters (consulta 2026-11-05); valores ilustrativos

Entrenar un modelo frente a servirlo

Para la mayoría de equipos el reto es servir

Entrenamiento

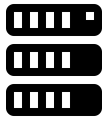
una vez, en un clúster grande



puntual

Inferencia

cada petición, con servidor con GPU



se repite 24/7 · costo continuo

Entrenar es puntual; servir es permanente.

GPU = unidad de procesamiento gráfico

Fuente: AWS Well-Architected, Generative AI Lens (docs.aws.amazon.com/wellarchitected/latest/generative-ai-lens)

Los pesos ocupan la memoria primero

Lo que sobra atiende a los usuarios

Memoria de una GPU de 24 GB



Cálculo

8 000 millones de parámetros × 2 bytes (16 bits) = **16 GB de pesos**

Lo libre atiende usuarios

- Usuario 1: su contexto ocupa VRAM
- Usuario 2: su contexto ocupa VRAM
- Usuario 3: su contexto ocupa VRAM

Más usuarios, menos memoria libre.

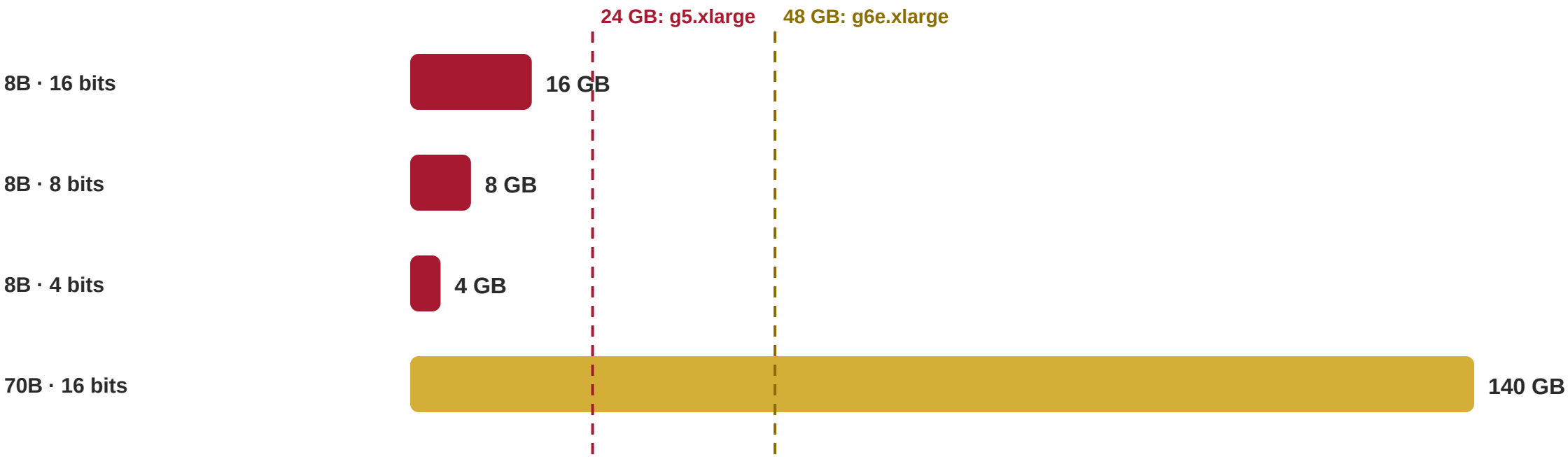
Regla práctica de dimensionamiento (cálculo propio).

GPU = unidad de procesamiento gráfico · GB = gigabyte · VRAM = memoria de la GPU

Fuente: aws.amazon.com/ec2/instance-types/g5/ (24 GB por GPU A10G); NVIDIA, Deep Learning Performance (2 bytes por parámetro en FP16); cálculo propio

Cuánta memoria necesita un modelo

Menos bits, menos GB de pesos



Pesos (GB) ≈ miles de millones de parámetros × bytes por parámetro

140 GB no caben en una GPU.

GPU = unidad de procesamiento gráfico

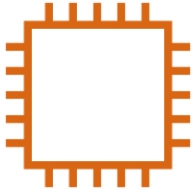
Fuente: aws.amazon.com/ec2/instance-types/ (g5: 24 GB; g6e: 48 GB por GPU); NVIDIA, Deep Learning Performance; cálculo propio

Tres formas de usar un modelo

Quién opera el modelo define el costo y el riesgo



API gestionada



Modelo abierto en EC2



Local en tu equipo



Tú operas menos

AWS opera el servidor

Tú operas la GPU

Tú operas todo

Los datos no salen del equipo

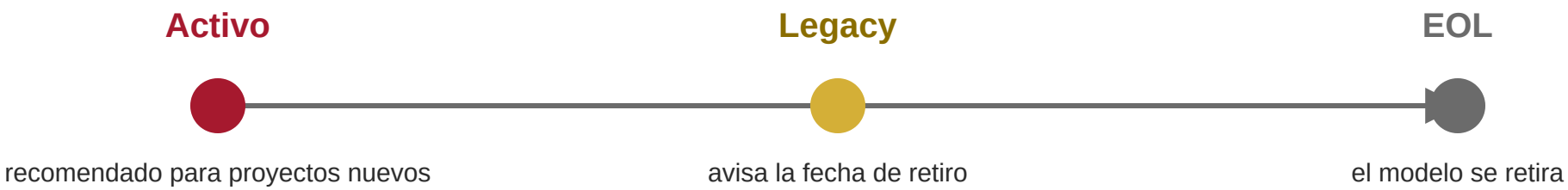
Más control = más operación a tu cargo.

API = interfaz de programación · EC2 = cómputo elástico de Amazon · AWS = Amazon Web Services · GPU = unidad de procesamiento gráfico

Fuente: Amazon Bedrock, Models at a glance (consulta 2026-11-05); Ollama (github.com/ollama/ollama)

Activo, Legacy y EOL: cada estado obliga a actuar

Activo, Legacy y EOL: cada estado cambia lo que el proyecto debe hacer



Claude Sonnet 5.5
activo: referencia para nuevos proyectos

Claude 3.5 Sonnet
en estado Legacy: planificar su reemplazo

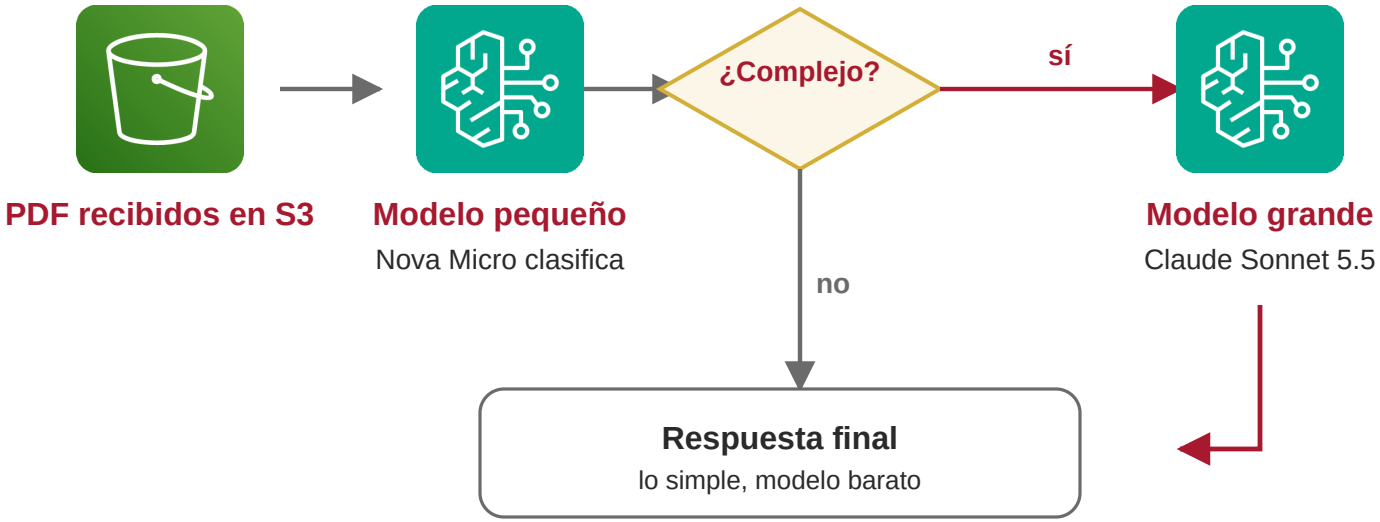
EOL = fin de vida del modelo

Fuente: docs.aws.amazon.com/bedrock/latest/userguide/model-lifecycle-legacy.html (ciclo de vida de modelos)

PATRÓN DE CASCADA

Un modelo pequeño filtra, uno grande resuelve

Caso ilustrativo de clasificación documental



Lo difícil sube

Solo lo complejo llega al modelo grande, que cuesta más por token.

PDF = formato de documento portátil

Fuente: Amazon Bedrock, Models at a glance y Pricing (consulta 2026-11-05); caso ilustrativo

Cuatro ideas para llevarse

Del mecanismo a la decisión de arquitectura



1 · Token a token

la latencia crece con la salida



2 · Memoria

decide el tamaño del servidor



3 · Muestreo

temperatura y Top-P



4 · Cascada

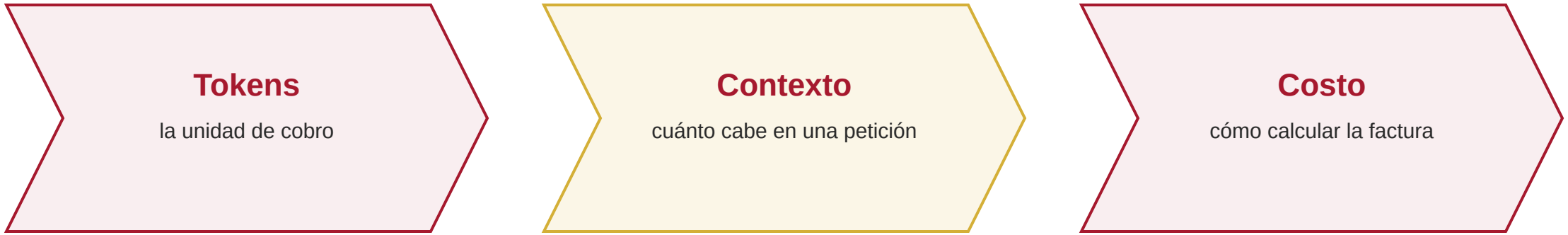
modelo pequeño primero

Cuatro decisiones que se repetirán en el resto de la clase.

Fuente: síntesis de las diapositivas 2 a 13 de esta parte

Siguiente: tokens, contexto y costo

De lo que el modelo hace a lo que se factura



Parte 2: cuánto cuesta un modelo y cómo medirlo.

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 2 de 4 · Tokens, Ventana de Contexto y Economía Cloud

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 2 de 4 · Tokens, Ventana de Contexto y Economía Cloud
IA = inteligencia artificial

De caracteres a subpalabras (tokens)

Cómo el texto se convierte en identificadores numéricos

Texto original

descentralización

Paso 1 · segmentar en subpalabras

des	central	iza	ción
prefijo	raíz	sufijo verbal	sufijo nominal

Paso 2 · un ID por trozo

ID des | ID central | ID iza | ID ción

El modelo procesa números, no letras.

ID = identificador

Fuente: Sennrich et al. (2016), arXiv:1508.07909 (codificación por pares de bytes)

Byte-Pair Encoding (BPE)

Fusión de pares frecuentes hasta alcanzar el vocabulario

Nivel 0 · caracteres

`c` `a` `s` `a` `s`

Paso 1 · fusionar «a + s»

`c` `as` `as`

Resultado: tres tokens, sin palabras desconocidas

Se repite hasta el tamaño de vocabulario.

Fuente: Sennrich et al. (2016), arXiv:1508.07909

El impuesto de tokenización en español

Mismo significado, distinto número de tokens

Inglés · «Deploy a private VPC»

Deploy | a | private | VPC — **4 tokens**

Español · «Despliega una VPC privada»

Des | plie | ga | una | VPC | priv | ada — **7 tokens**

Más trozos: más tokens facturados.

Para presupuestar, medir el ratio real.

VPC = red virtual privada

Fuente: ejemplo ilustrativo de tokenización (conteo no medido)

Menos fragmentación, capa de salida más grande

Menos fragmentación a costa de una capa de salida más grande

Llama 2 · 2023



32.000 tokens

Fragmentación severa en español

Llama 3 · 2024



128.256 tokens

Equilibrio tokens y memoria

Compensación física

Más VRAM en la capa de salida.

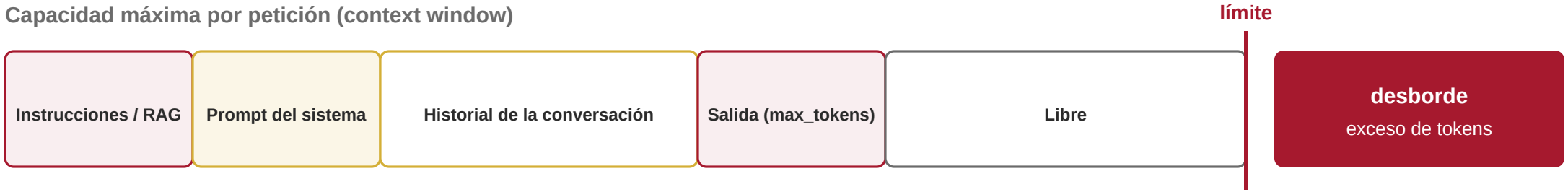
VRAM = memoria de la GPU

Fuente: Meta, model cards de Llama 2 y Llama 3 (llama.meta.com)

Cómo se reparte el espacio disponible

Cómo se reparte el espacio y qué ocurre al desbordarse

Capacidad máxima por petición (context window)



Si el total supera el límite
error HTTP 400 o truncamiento silencioso

Pérdida de instrucciones iniciales
lo primero que se recorta puede ser clave

Mitigación en producción

- Ventana deslizante
- Resumen periódico del historial
- Monitoreo de tokens con el SDK

Cada token suma costo y latencia.

RAG = generación aumentada por recuperación · HTTP = protocolo web · SDK = kit de desarrollo de software

Fuente: AWS Bedrock Converse API y guía de cuotas (docs.aws.amazon.com/bedrock)

EVOLUCIÓN TEMPORAL

De 2.048 a 200.000 tokens de contexto

Un salto de casi cien veces en cuatro años

2020

GPT-3

2.048 tokens

Solo párrafos cortos

2024

Claude 3.5 Sonnet

200.000 tokens

libros completos y bases de código

Cálculo: 200.000 ÷ 2.048 ≈ 98 veces más contexto en cuatro años.

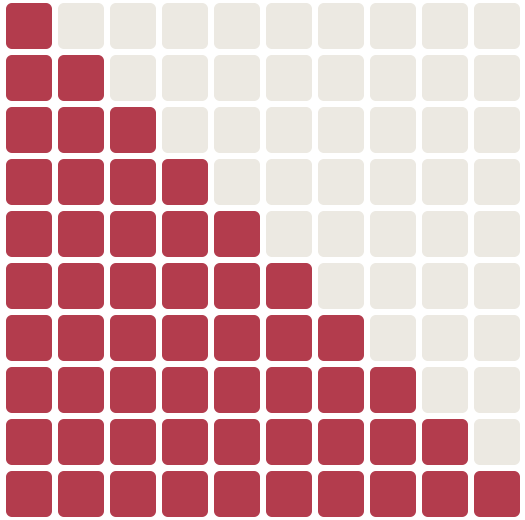
GPT = transformador generativo preentrenado

Fuente: Brown et al. (2020), arXiv:2005.14165 (GPT-3); Anthropic, documentación de modelos (Claude 3.5 Sonnet)

Por qué el contexto largo gasta memoria

Por qué el contexto largo consume memoria de la GPU

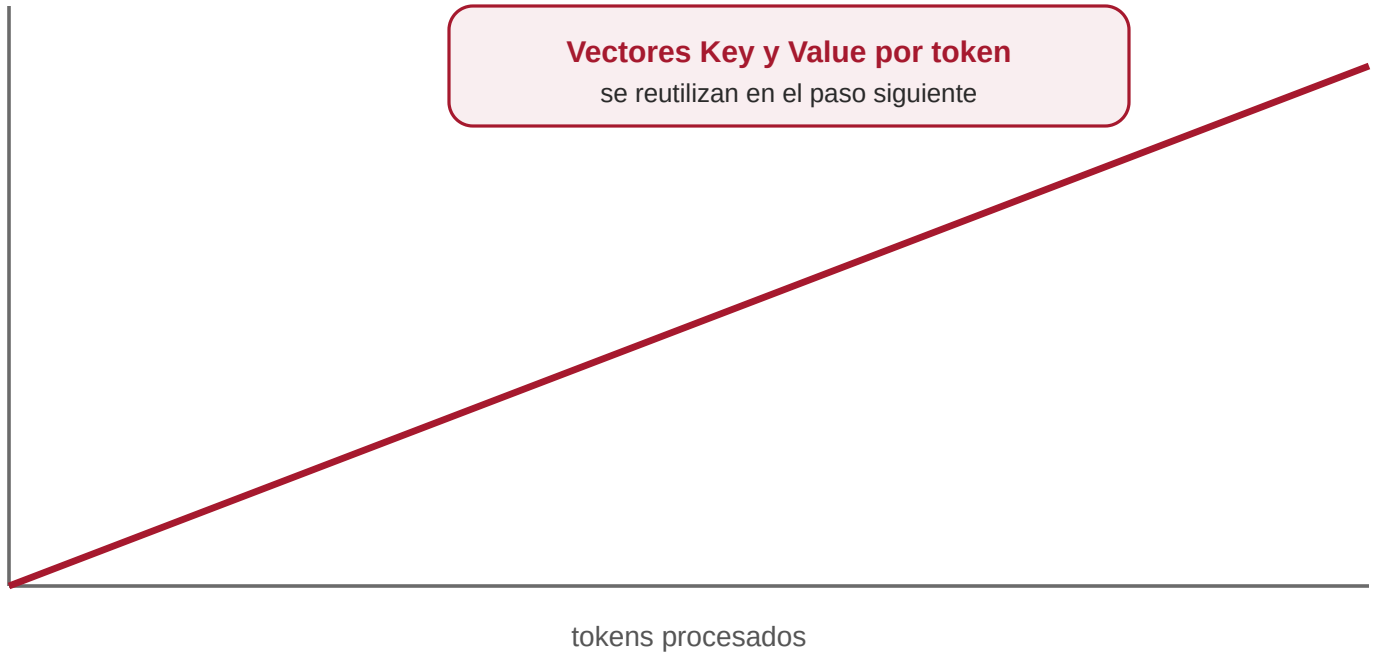
Cada token mira a cada token



N × N pares de tokens

N = 100.000 → 10^{10} pares (N^2)

KV Cache: crece con cada token



KV = clave-valor (caché de atención) · GPU = unidad de procesamiento gráfico

Fuente: Dao et al. (2022), FlashAttention (arXiv:2205.14135)

El prefijo compartido se procesa una vez

Descuento en contextos compartidos: el prefijo se procesa una vez

Ejemplo: manual de 20.000 tokens

1.ª petición · Cache write

manual 20.000 tokens

Escritura: $2.50 / 1M tokens

GPU: guarda el KV Cache

reutiliza →

2.ª a n.ª · Cache read

manual 20.000 tokens

Lectura: $0.10 / 1M tokens

Descuento de 95 %

Regla: prefijo al inicio

Cambiar el inicio invalida la caché.

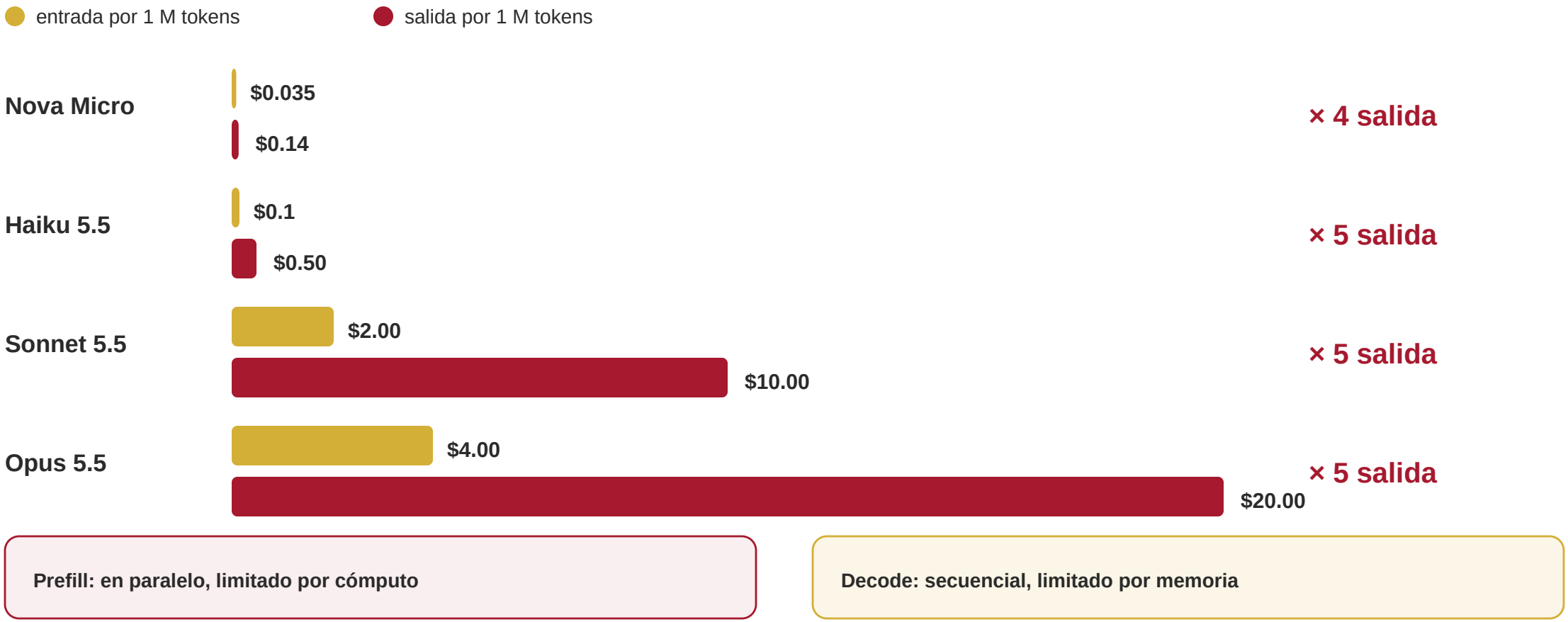
TTL de la caché: 5 minutos.

GPU = unidad de procesamiento gráfico · KV = clave-valor (caché de atención) · TTL = tiempo de vida de la caché

Fuente: docs.aws.amazon.com/bedrock/latest/userguide/prompt-caching.html; tarifas de Bedrock (consulta 2026-11-05)

Entrada frente a salida: la salida cuesta 4 a 5 veces más

Prefill es paralelo; decode genera un token por paso



Fuente: tarifas de Amazon Bedrock us-east-1 (consulta 2026-11-05); cálculo propio de la razón salida/entrada

Tarifas en Amazon Bedrock (noviembre 2026)

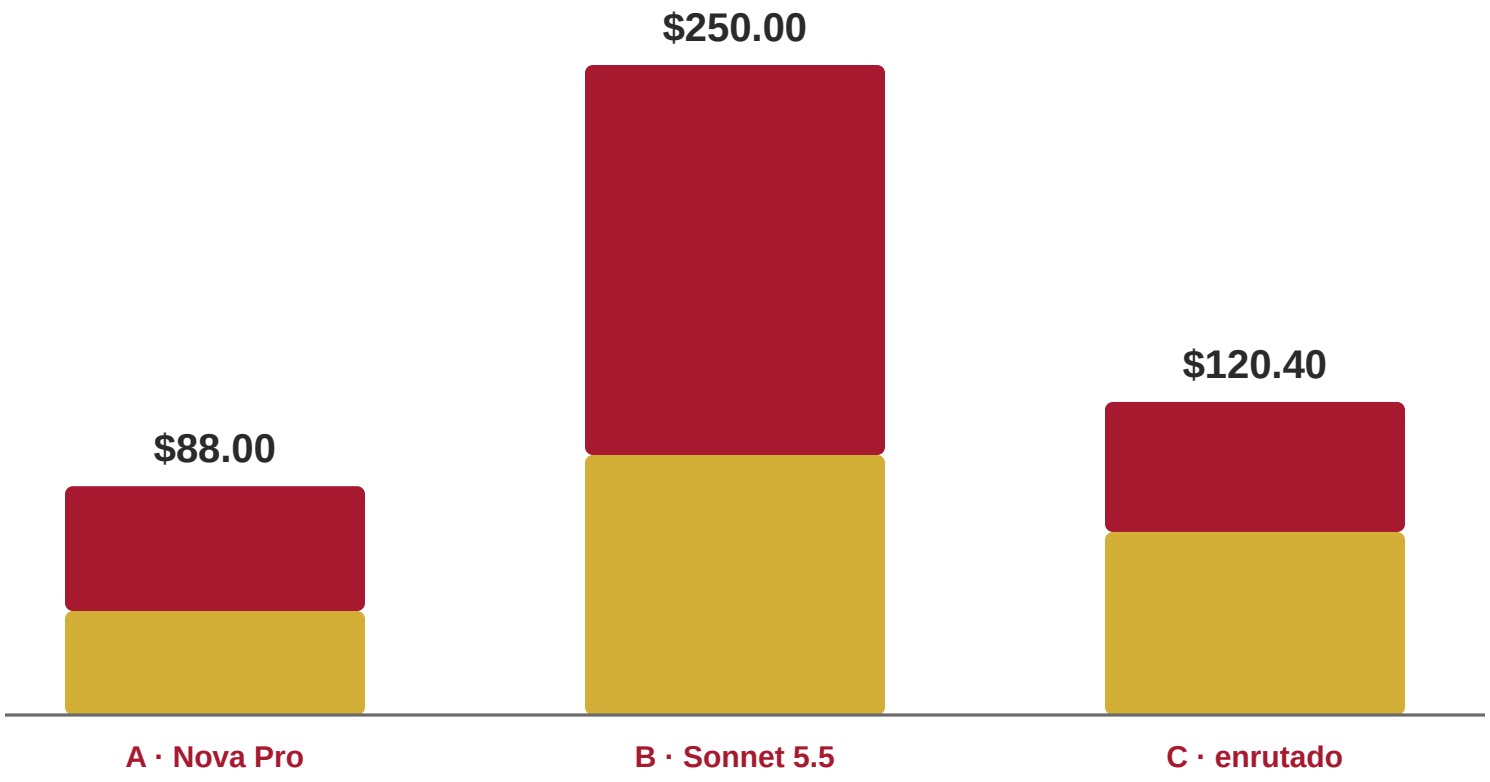
Precio por millón de tokens, región us-east-1

Modelo	Entrada / 1 M	Salida / 1 M
Amazon Nova Micro	$0.035	$0.14
Amazon Nova Lite	$0.06	$0.24
Amazon Nova Pro	$0.80	$3.20
Claude Haiku 5.5	$0.10	$0.50
Claude Sonnet 5.5	$2.00	$10.00
Claude Opus 5.5	$4.00	$20.00

Fuente: aws.amazon.com/bedrock/pricing/ (us-east-1, consulta 2026-11-05)

Estimación de costos mensuales paso a paso

Escenarios con 50 000 consultas al mes



Supuesto

50 000 consultas al mes
1 000 tokens de entrada
300 tokens de salida

Volumen: 50 M entrada
y 15 M salida al mes

Enrutar ahorra 52 %
frente a usar solo Sonnet 5.5 (cálculo propio)

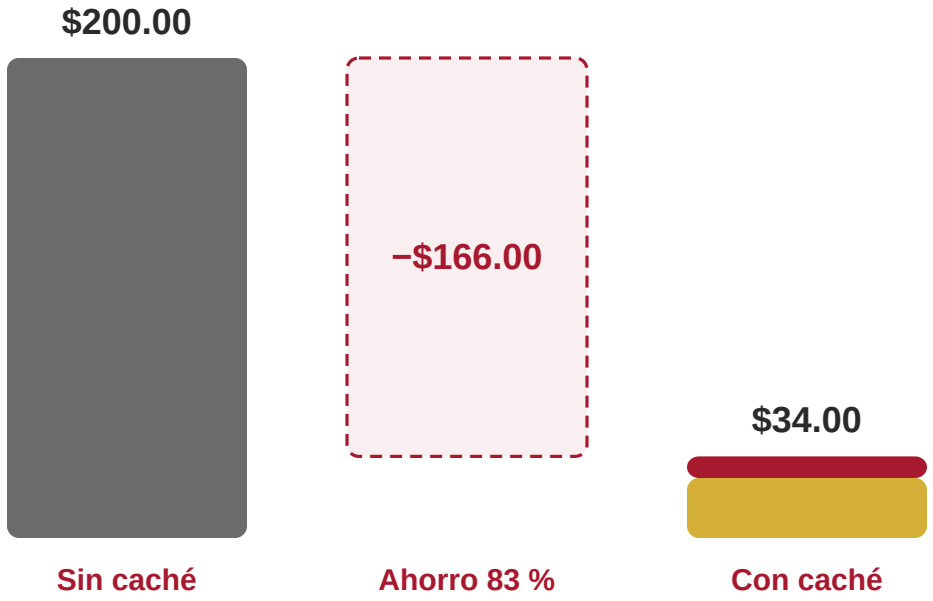
Reparto 80/20: supuesto de arquitectura.

Tarifas: aws.amazon.com/bedrock/pricing/ (us-east-1, consulta 2026-11-05); cálculo propio.

Fuente: tarifas de Bedrock us-east-1 (consulta 2026-11-05); cálculo propio

Con 90 % de aciertos: de $200 a $34 al mes

Con 90 % de aciertos, la entrada baja de $200 a $34 al mes



- 10 M × $2.50 = $25.00
- 90 M × $0.10 = $9.00

100 M × $2.00 = $200.00

Mismo modelo, misma respuesta
la caché cambia el costo, no la calidad

Fuente: tarifas de Bedrock para Claude Sonnet 5.5 (consulta 2026-11-05); cálculo propio con supuesto de 90 % de aciertos

Controlar costos y latencia

Directrices para controlar costos y latencia

1. **Prefijo fijo**
   lo estable, al inicio
2. **max_tokens**
   tope estricto de salida
3. **Medir el español**
   audita el ratio real
4. **Enrutar la carga**
   modelo pequeño primero

Fuente: AWS Well-Architected, Generative AI Lens (arquitecturas de producción)

Siguiente: el servidor de inferencia en la nube

Cómo se despliegan, monitorean y escalan los modelos



Motores de servicio

vLLM dentro de contenedores

Métricas

TTFT, TPOT y throughput

Hardware de GPU

A10G, L40S en instancias EC2

Parte 3: del modelo al servidor.

vLLM = motor de inferencia de código abierto · TTFT = tiempo hasta el primer token · TPOT = tiempo por token de salida · GPU = unidad de procesamiento gráfico · EC2 = cómputo elástico de Amazon

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

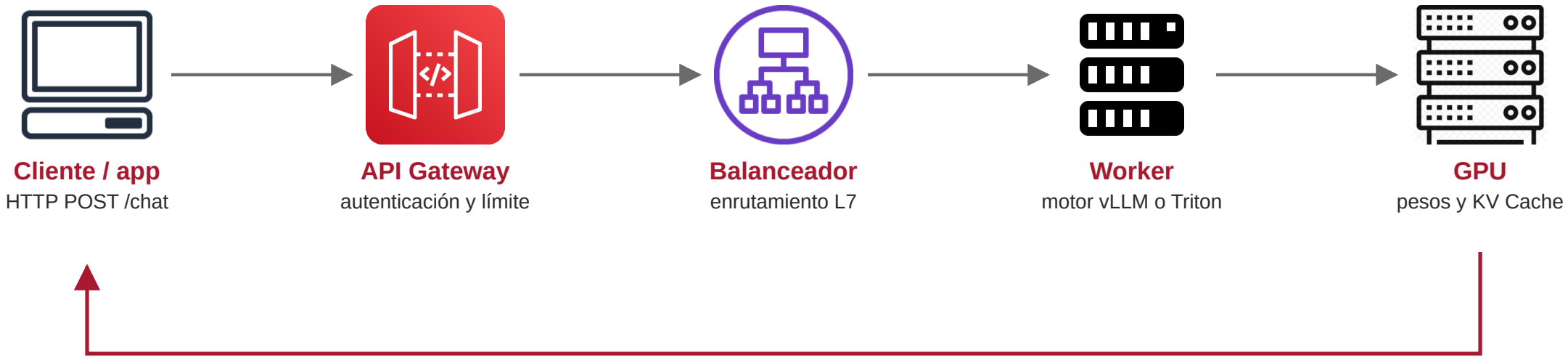
Parte 3 de 4 · El Servidor de Inferencia, Motores y Métricas de Rendimiento

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 3 de 4 · El Servidor de Inferencia, Motores y Métricas de Rendimiento
IA = inteligencia artificial

Flujo integral de una petición de inferencia

Del cliente HTTP al acelerador de hardware



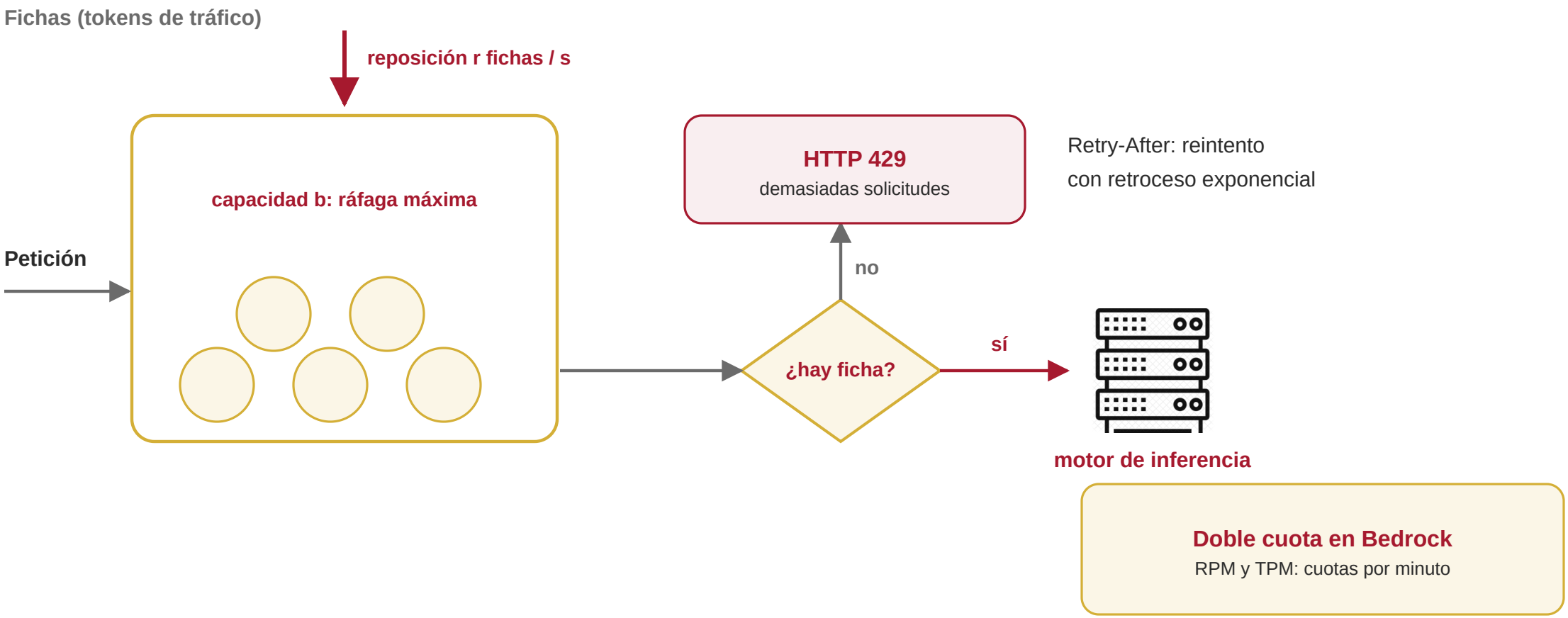
Desacopla la autenticación del cómputo

HTTP = protocolo web · POST = envío de datos al servidor · API = interfaz de programación · L7 = capa 7 (aplicación) · vLLM = motor de inferencia de código abierto · GPU = unidad de procesamiento gráfico · KV = clave-valor (caché de atención)

Fuente: AWS Well-Architected Framework, Machine Learning Lens (Architecture Topologies)

Protección contra sobrecarga: token bucket

La capa perimetral evita saturar los aceleradores

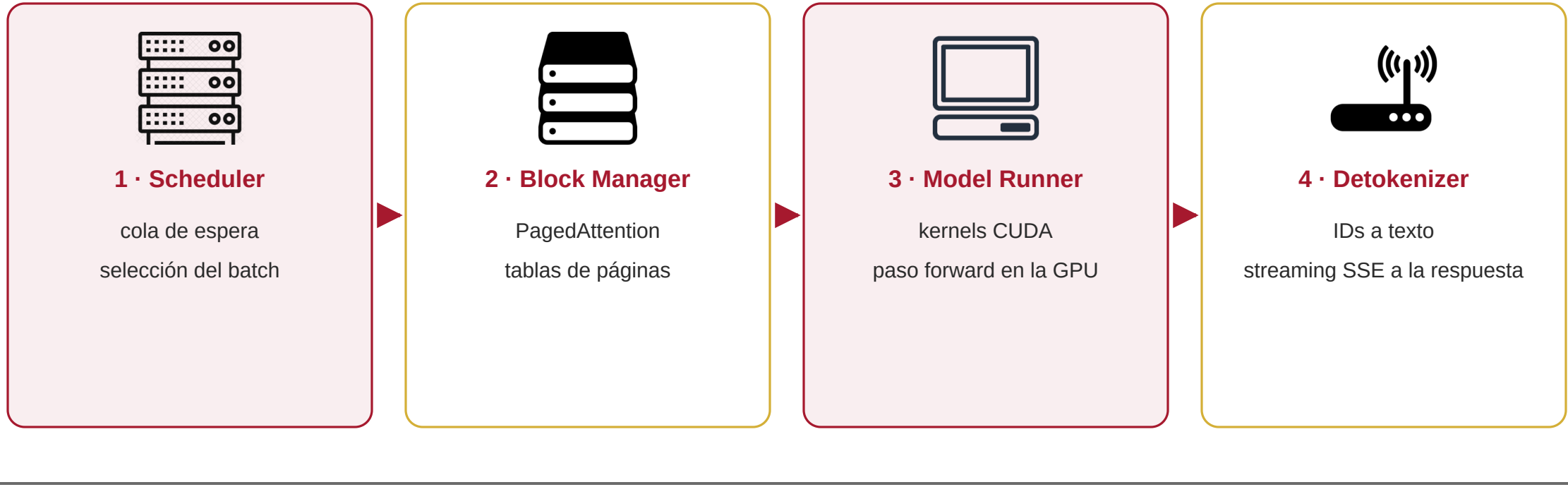


HTTP = protocolo web · RPM = peticiones por minuto · TPM = tokens por minuto

Fuente: AWS Bedrock, Quotas and Limits; API Gateway, limitación de tasa (documentación oficial)

Anatomía interna de un motor de inferencia

Cuatro subsistemas que mantienen la GPU ocupada



Objetivo: GPU ocupada, VRAM sin fragmentar.

CUDA = plataforma de cómputo paralelo para GPU · GPU = unidad de procesamiento gráfico · SSE = eventos enviados por el servidor · VRAM = memoria de la GPU

Fuente: Kwon et al. (2023), PagedAttention (vLLM); Yu et al. (2022), Orca

Agrupar por iteración, no por lote

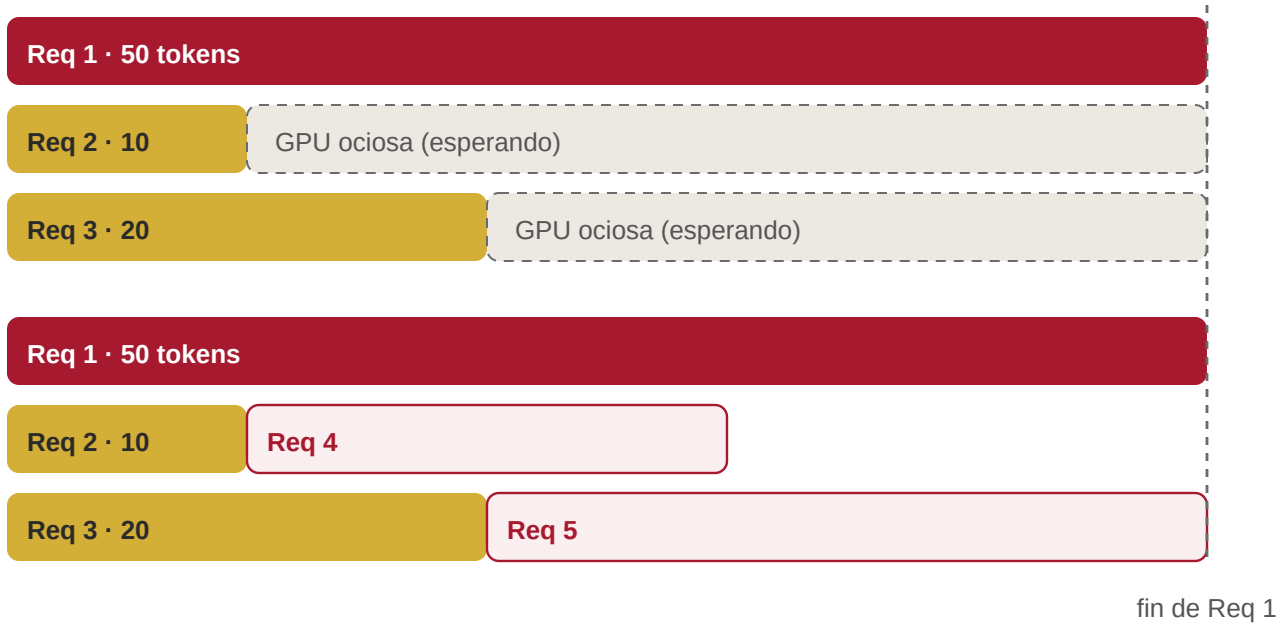
Agrupar a nivel de iteración en lugar de por lote

Batch estático

se espera al lote completo

Continuous batching

entra otra al terminar



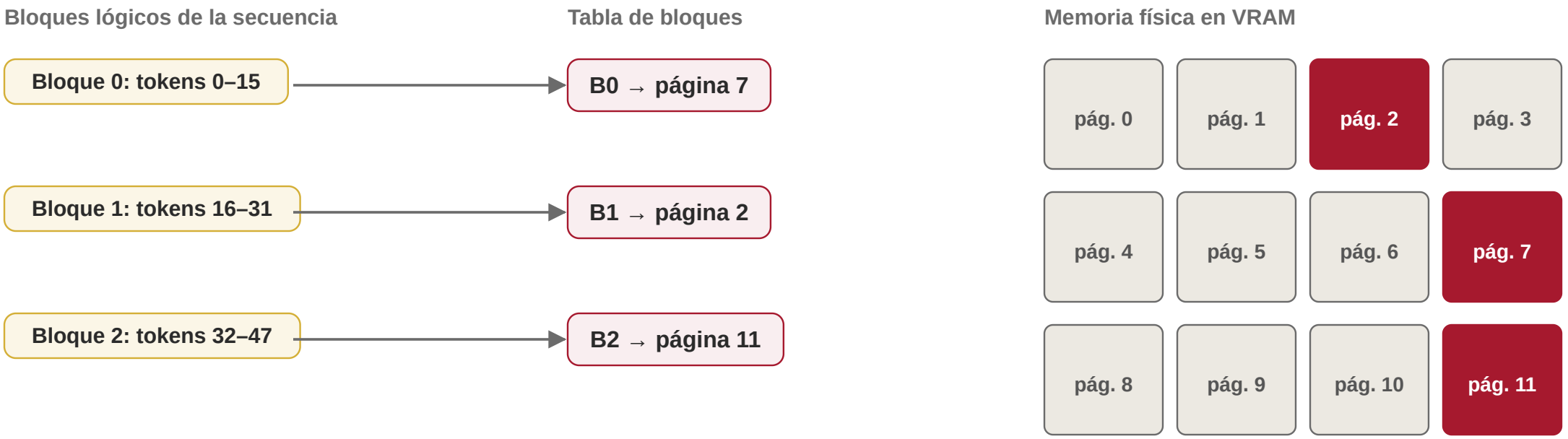
Longitudes ilustrativas, no medidas.

GPU = unidad de procesamiento gráfico

Fuente: Yu et al. (2022), Orca: A Distributed Serving System for Transformer-Based Generative Models

Bloques lógicos a páginas físicas

Bloques lógicos mapeados a páginas físicas, como en un sistema operativo



Lectura continua para el modelo

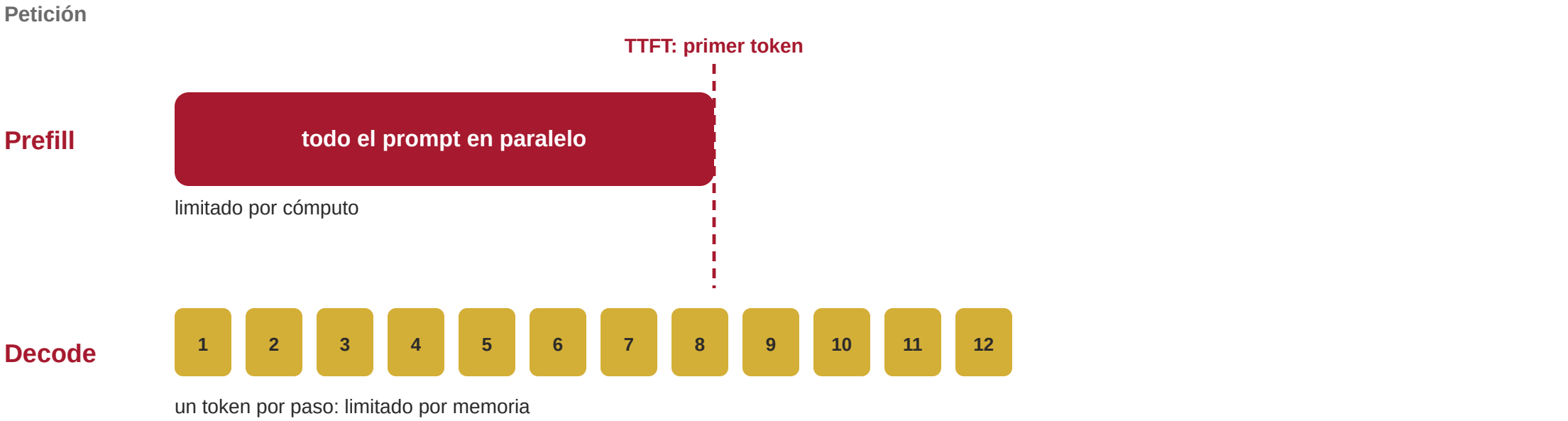
Páginas no contiguas, cero fragmentación.

VRAM = memoria de la GPU

Fuente: Kwon et al. (2023), PagedAttention (arXiv:2309.06180)

De cómputo a ancho de banda

De saturar núcleos de cálculo a saturar ancho de banda



Prefill: se paga una vez

Decode: se paga por token

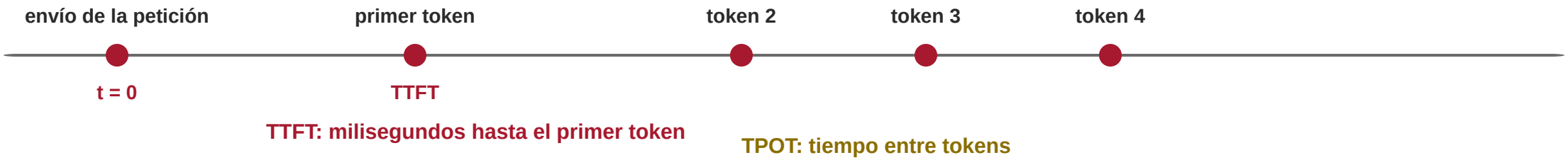
Por eso la salida cuesta más.

TTFT = tiempo hasta el primer token

Fuente: Patel et al. (2024), Splitwise: Efficient Generative LLM Inference Using Phase Splitting

Lo que define el SLA de inferencia

Los indicadores que definen el acuerdo de nivel de servicio (SLA)



TTFT

primer token: lo que el usuario percibe

TPOT

velocidad al generar cada token

Throughput

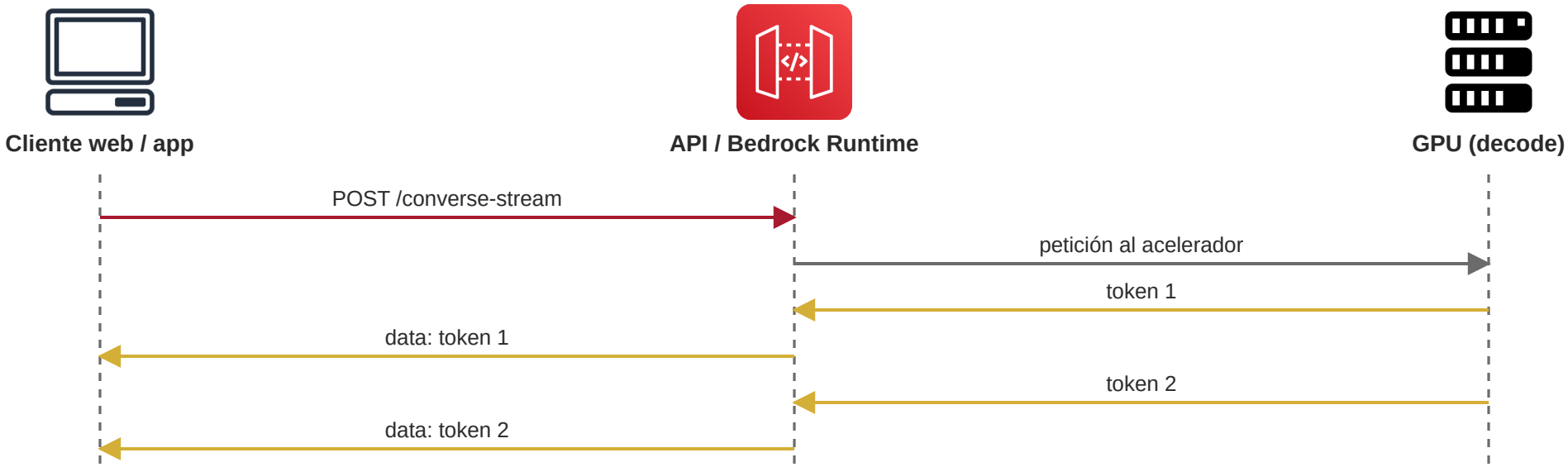
tokens por segundo de toda la flota

TTFT = tiempo hasta el primer token · TPOT = tiempo por token de salida

Fuente: AWS Bedrock, documentación de monitoreo (docs.aws.amazon.com/bedrock/latest/userguide/monitoring-cw.html)

Token por token mejora la percepción

Entregar token por token mejora la experiencia percibida



Sin streaming
pantalla vacía hasta el final

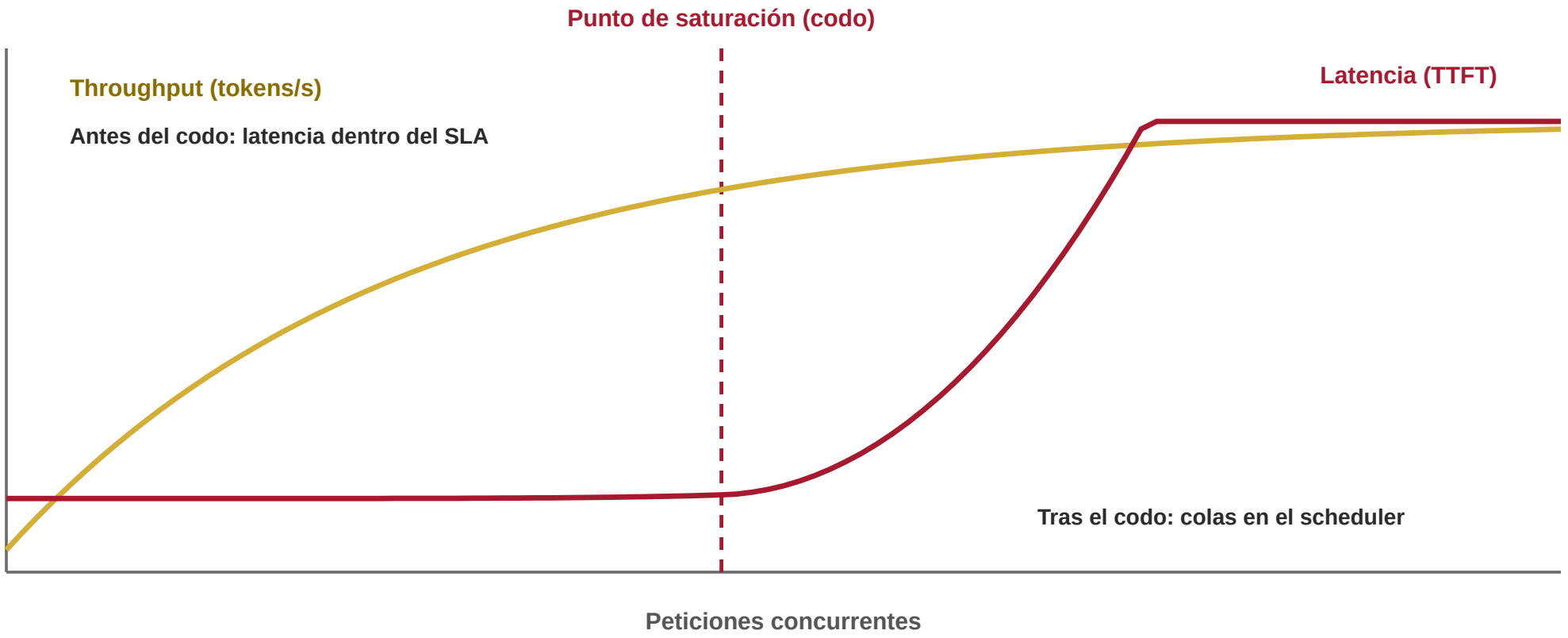
Con SSE (text/event-stream)
el primer token llega enseguida

API = interfaz de programación · GPU = unidad de procesamiento gráfico · POST = envío de datos al servidor · SSE = eventos enviados por el servidor

Fuente: AWS Bedrock, ConverseStream API; W3C, Server-Sent Events

El punto óptimo antes del colapso

Dónde está el punto óptimo antes de que la latencia colapse



Curva cualitativa, no medida.

SLA = acuerdo de nivel de servicio

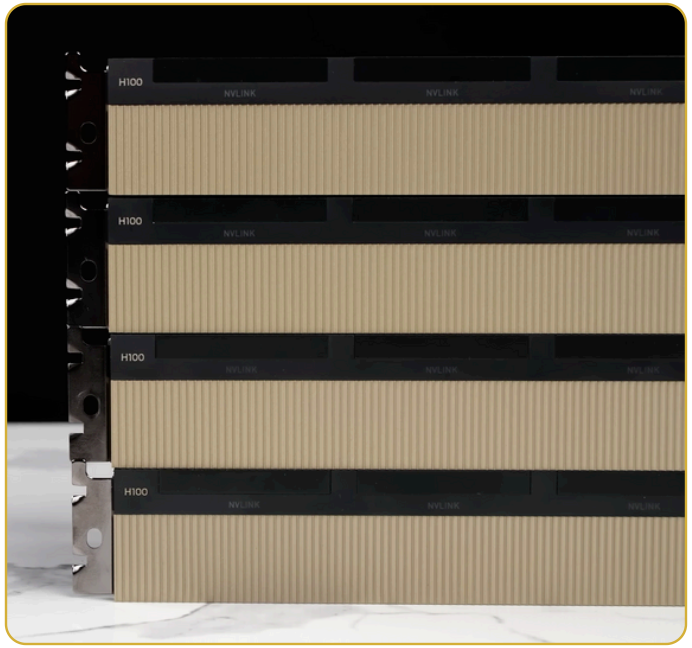
Fuente: curva cualitativa ilustrativa; métricas en la documentación de monitoreo de Bedrock

Precio on-demand por hora

Precio on-demand por hora para inferencia dedicada

Instancia	GPU	Memoria de GPU	Precio
g5.xlarge 8B cuantizado	1 × A10G	24 GB	$1.006 / h
g6e.xlarge Modelos grandes, una GPU	1 × L40S	48 GB	$1.861 / h
p5.48xlarge Modelos de frontera: clúster de GPU	8 × H100 (80 GB c/u)	640 GB	$55.04 / h

Precio por hora, us-east-1.

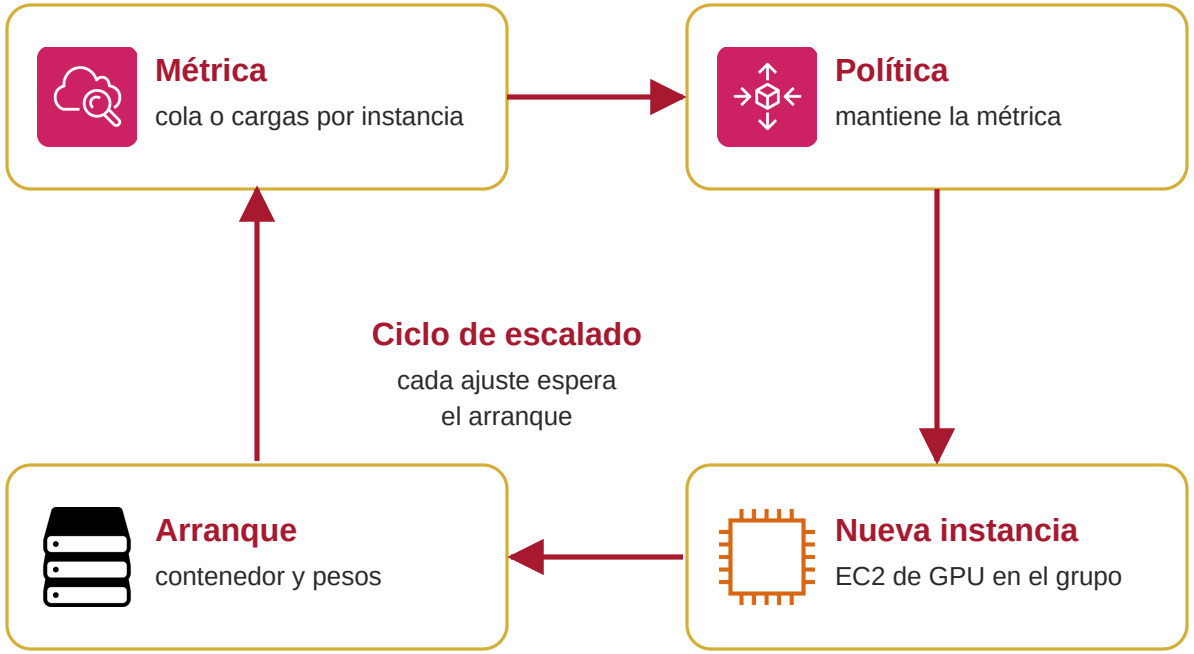


GPU = unidad de procesamiento gráfico · GB = gigabyte

Fuente: AWS Price List, EC2 us-east-1 (consulta 2026-11-05) · Foto: Geekerwan, Wikimedia Commons (CC BY 3.0)

Target tracking y arranque en frío

Target tracking y mitigación del arranque en frío (cold start)



Arranque en frío (cold start)
cargar pesos toma minutos

Mitigaciones:

- volumen EBS con pesos precargados
- imagen de máquina con el modelo
- instancias calientes en el mínimo

EC2 = cómputo elástico de Amazon · GPU = unidad de procesamiento gráfico · EBS = volúmenes de bloques elásticos

Fuente: AWS Auto Scaling, target tracking; EC2 Auto Scaling

Latencia, cuotas y auditoría

Latencia, cuotas y auditoría de prompts en S3

Métricas en Amazon CloudWatch

InvocationLatency
latencia de cada llamada

Invocations
conteo de llamadas

InvocationThrottles
rechazos por cuota (429)

Input/OutputTokenCount
tokens facturables



Alarma
InvocationThrottles

Registro de peticiones (auditoría)



Model Invocation Logging
prompt y respuesta en S3



cifrado con KMS
claves del cliente

Auditoría y seguridad
texto íntegro, acceso restringido

KMS = servicio de gestión de claves

Fuente: AWS Bedrock, documentación de monitoreo (CloudWatch y Model Invocation Logging)

Cuatro fallas clásicas en inferencia y cómo mitigarlas

De falta de memoria a picos de latencia en percentiles altos

Falla	Causa típica	Mitigación
1 · Error OOM en GPU	VRAM llena de pesos y contextos	**limitar contexto; cuantizar**
2 · Cold start lento	minutos para cargar pesos	**instancias calientes; pesos en volumen precargado**
3 · Throttling 429	cuotas RPM o TPM rebasadas	**cola asíncrona y reintentos con retroceso**
4 · P99 inestable	interferencia entre usuarios	**límites por cliente y batch controlado**

OOM = falta de memoria · GPU = unidad de procesamiento gráfico · VRAM = memoria de la GPU · RPM = peticiones por minuto · TPM = tokens por minuto

Fuente: AWS Well-Architected, Machine Learning Lens (pilar de confiabilidad)

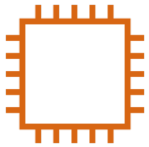
Siguiente: la gran decisión de arquitectura

Servicio gestionado, servidor propio o modelo local



APIs gestionadas

Amazon Bedrock: serverless, pagas por token



Servidor propio

EC2 con GPU: pagas por hora



Inferencia local

Ollama: tú operas todo

Parte 4: criterios técnicos y regulatorios.

EC2 = cómputo elástico de Amazon · GPU = unidad de procesamiento gráfico

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

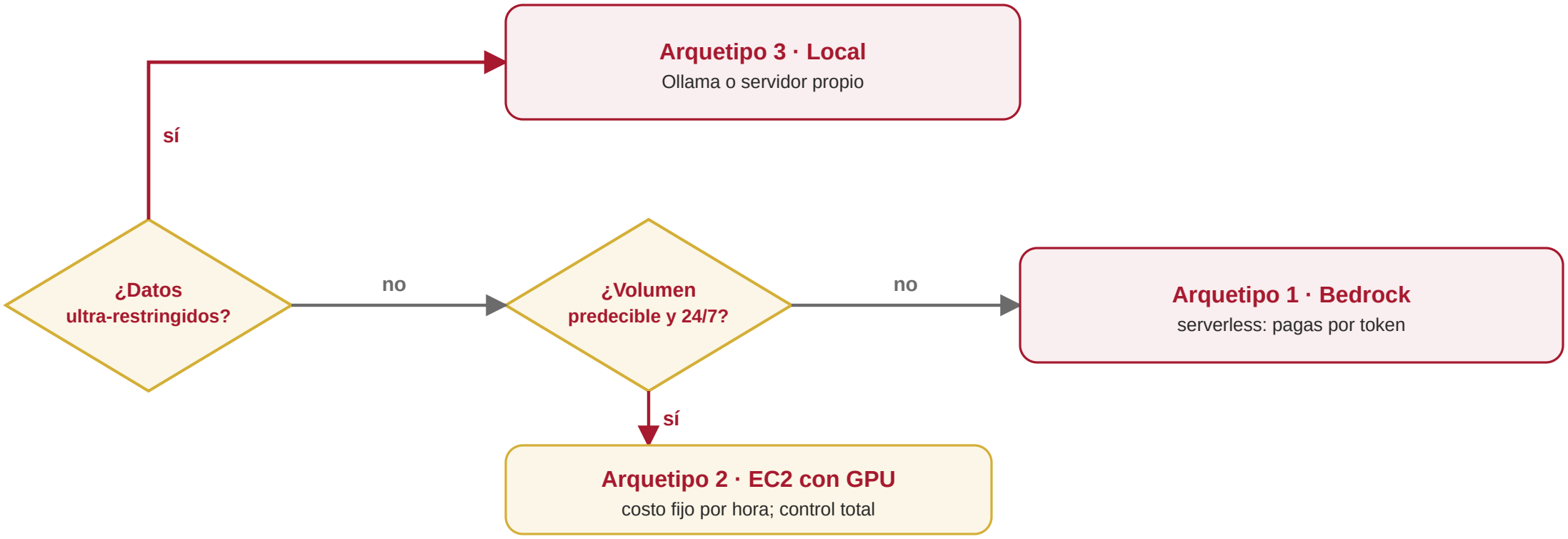
Parte 4 de 4 · Decisiones de Despliegue, Seguridad y Marco Regulatorio

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 4 de 4 · Decisiones de Despliegue, Seguridad y Marco Regulatorio
IA = inteligencia artificial

Árbol de decisión: tres caminos

Un árbol de decisión entre API gestionada, servidor propio y local



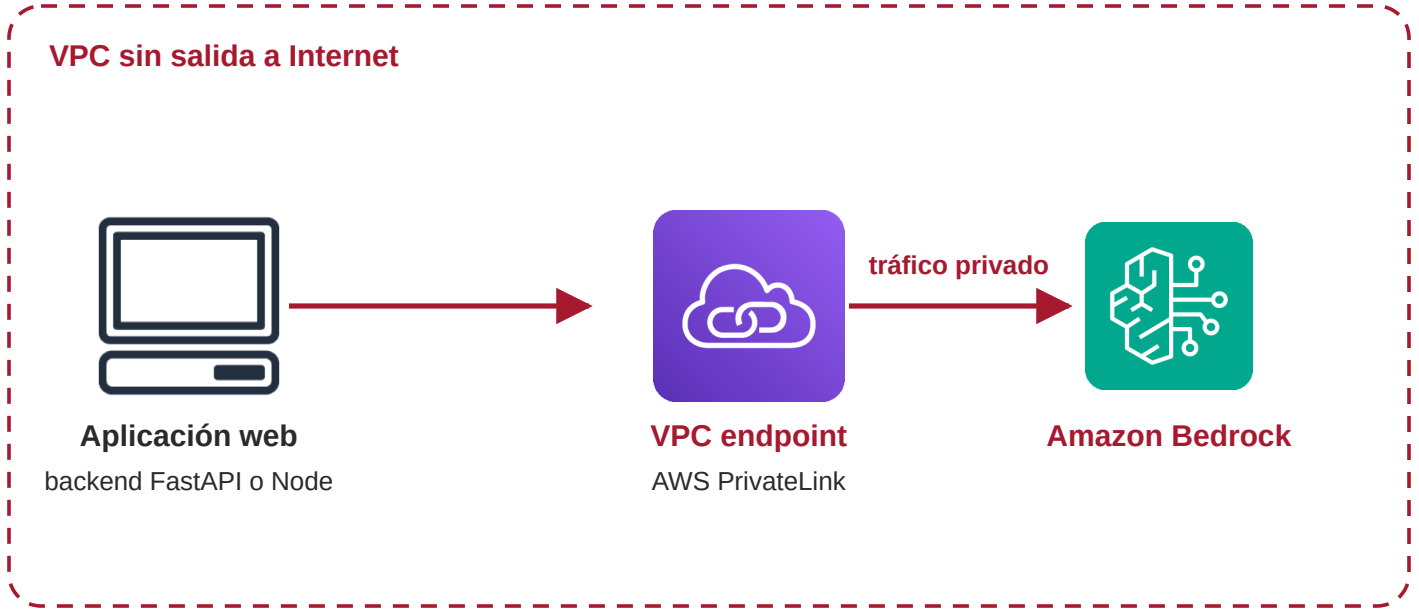
Los datos sensibles pasan primero: decide la regulación, no el precio.

EC2 = cómputo elástico de Amazon · GPU = unidad de procesamiento gráfico · API = interfaz de programación

Fuente: AWS Decision Guide for Generative AI; AWS Well-Architected Framework

Máxima velocidad, red privada

Máxima velocidad de entrega y aislamiento en red privada



Sin Internet Gateway ni NAT
tráfico por la red interna de AWS

Guardrails antes del modelo
bloquean inyección y fuga de datos

Escalado elástico sin servidores.

Documentación de AWS PrivateLink y endpoints de Amazon Bedrock.

VPC = red virtual privada · AWS = Amazon Web Services · NAT = traducción de direcciones

Fuente: AWS Bedrock, documentación de seguridad y endpoints de VPC (AWS PrivateLink)

Velocidad frente a cuotas y control

Velocidad y cero costo ocioso frente a cuotas y control

Ventajas

- Factura $0 sin consultas
- Producción en horas, sin comprar hardware
- Modelos de frontera actualizados
- Guardrails, KMS e IAM integrados

Límites

- Costo por token alto con uso masivo
- Cuotas RPM y TPM: HTTP 429
- Dependencia de retiros del proveedor

KMS = servicio de gestión de claves · IAM = gestión de identidades y accesos · RPM = peticiones por minuto · TPM = tokens por minuto · HTTP = protocolo web

Fuente: AWS Well-Architected Framework, Machine Learning Lens (análisis de compromisos)

SERVIDORES DEDICADOS

Control total a cambio de costo fijo

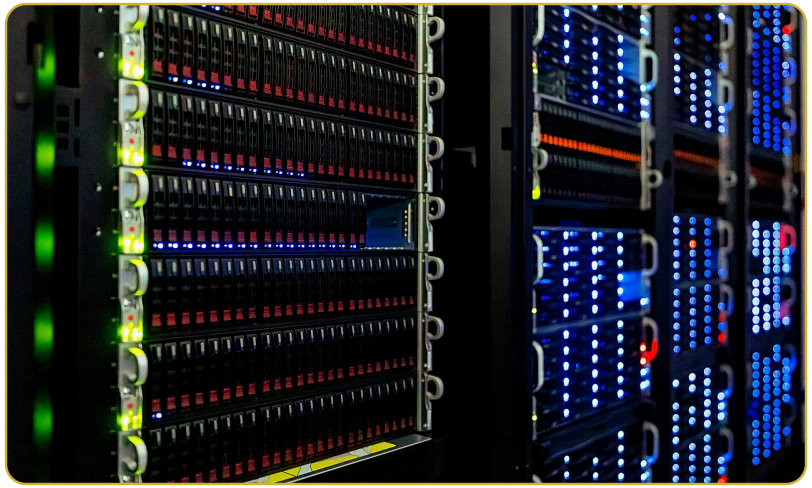
Control de la pila a cambio de un costo fijo y operación propia

EC2 g5 o g6e en VPC privada

- **GPU de 24 o 48 GB**
  pesos en VRAM local
- **Driver NVIDIA y CUDA**
  kernels en la GPU
- **Contenedor con vLLM**
  PagedAttention y batching
- **API compatible con OpenAI**
  /v1/chat/completions
- **Usuario / aplicación**
  cliente

Volumen EBS gp3

pesos de Ministral o gpt-oss



Soberanía técnica sobre cada hiperparámetro

Costo fijo, aunque nadie consulte.

API = interfaz de programación · vLLM = motor de inferencia de código abierto · NVIDIA = fabricante de GPU · CUDA = plataforma de cómputo paralelo para GPU · GPU = unidad de procesamiento gráfico · GB = gigabyte · VRAM = memoria de la GPU · EC2 = cómputo elástico de Amazon · VPC = red virtual privada · EBS = volúmenes de bloques elásticos

Fuente: AWS EC2 G5 y G6e; guía de vLLM · Foto: NOIRLab/NSF/AURA, Wikimedia Commons (CC BY 4.0)

Costo marginal bajo, costo fijo alto

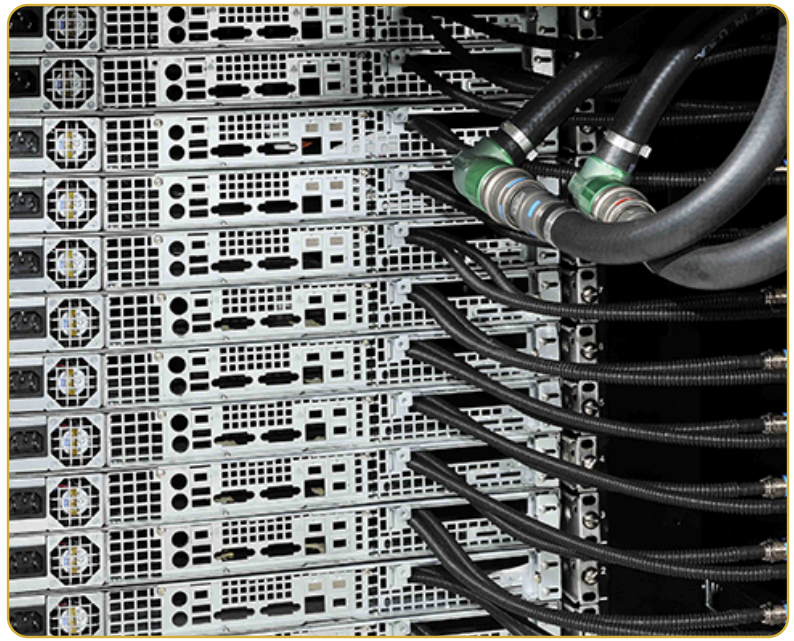
Costo marginal bajo frente a costo fijo y mantenimiento

Ventajas

- Costo marginal bajo a plena carga
- Sin cuotas impuestas por terceros
- Versión del modelo bajo control propio
- Aislamiento: sin multitenencia de terceros

Límites

- Costo fijo: $734 al mes
- Parches y drivers a cargo
- Arranque en frío de minutos



$1.006 / h × 730 h = $734.38 / mes (cálculo propio)

Precio on-demand g5.xlarge, us-east-1 (consulta 2026-11-05).

Fuente: FinOps Foundation; EC2 g5.xlarge (consulta 2026-11-05) · Foto: CoolIT, Wikimedia Commons (CC BY-SA 3.0)

Arquetipo 3: inferencia local y soberana con Ollama

Máxima confidencialidad sin salida de datos

Hardware local
GPU dedicada o RAM unificada

Ollama
motor en C++ (llama.cpp)

API REST local
servicio en el puerto 11434

GGUF cuantizado a 4 u 8 bits

Menos memoria; modelo local.

La app llama al puerto local.



Máxima confidencialidad
el dato nunca sale del equipo

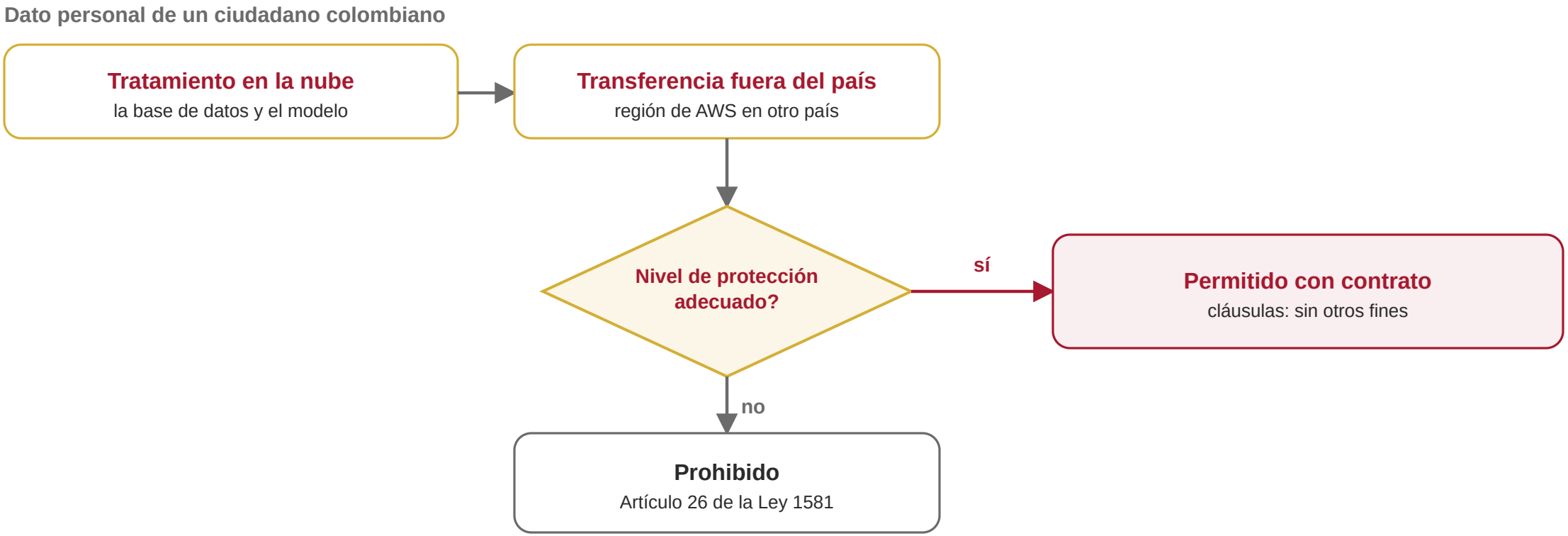
La organización opera todo.

GPU = unidad de procesamiento gráfico · RAM = memoria de acceso aleatorio · API = interfaz de programación · REST = transferencia de estado representacional · GGUF = formato de modelos cuantizados

Fuente: Ollama (github.com/ollama/ollama) · Foto: F. Hirzinger, CERN, Wikimedia Commons (CC BY-SA 3.0)

Soberanía del dato y regulación en Colombia

Cumplimiento de la Ley 1581 de 2012 en IA



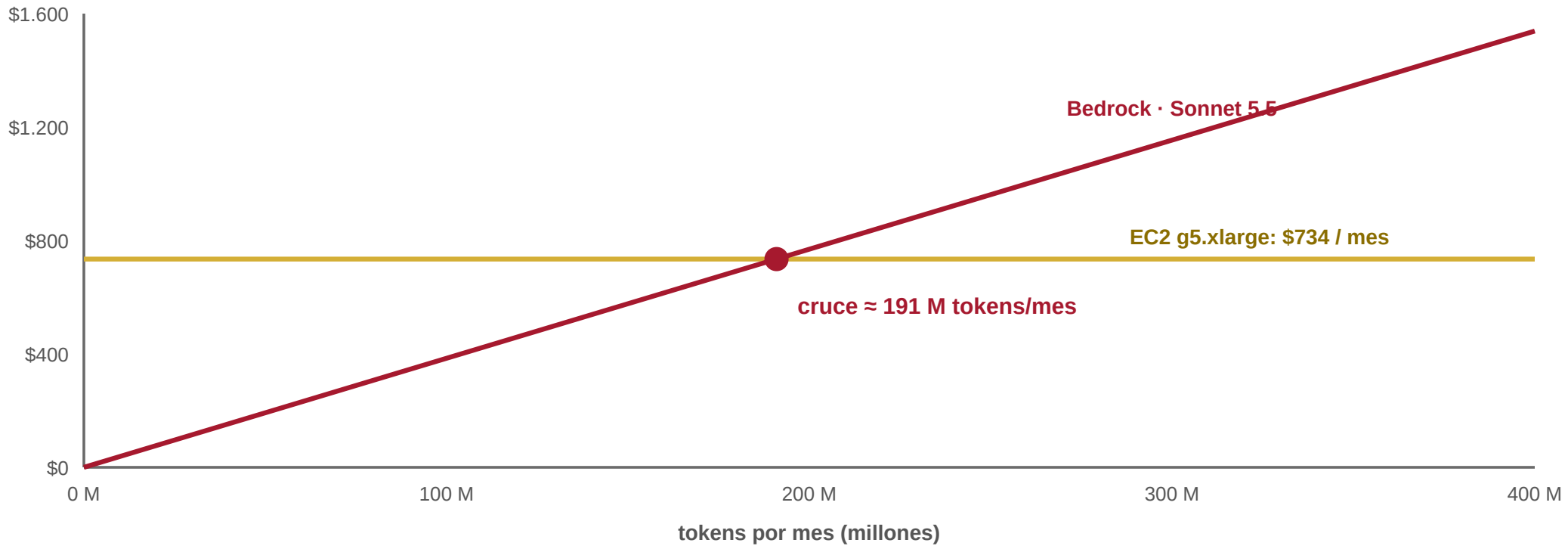
Verificar el nivel de protección de cada región.

AWS = Amazon Web Services · IA = inteligencia artificial

Fuente: Ley 1581 de 2012, art. 26 (Gestor Normativo, Función Pública); Superintendencia de Industria y Comercio

Costo fijo frente a costo variable

Costo fijo de una g5.xlarge frente a costo variable de Bedrock



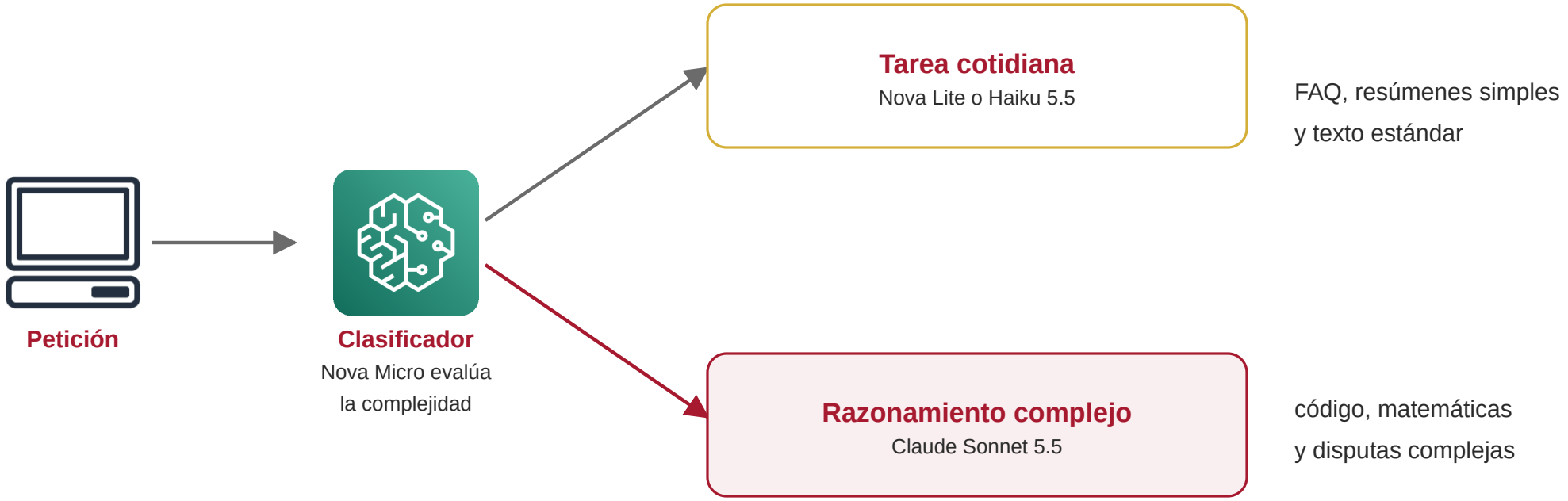
Bajo el cruce conviene la API; por encima, el servidor dedicado.

EC2 = cómputo elástico de Amazon · API = interfaz de programación

Fuente: cálculo propio con tarifas de Sonnet 5.5 (Bedrock, us-east-1) y g5.xlarge a $1.006 / h × 730 h

Arquitectura híbrida: el enrutador inteligente

Clasificar la consulta antes de elegir el modelo



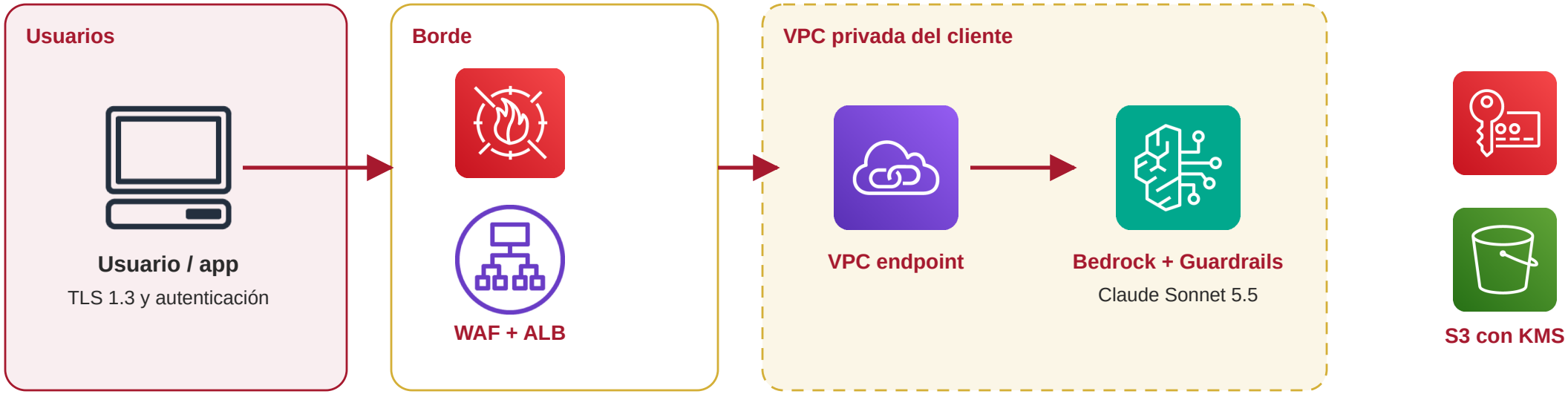
El enrutador usa un modelo barato para decidir y solo escala lo difícil.

FAQ = preguntas frecuentes

Fuente: AWS Generative AI Lens (Architecture Patterns: model routing and cascading)

Topología de referencia para IA en producción

Red privada, inspección perimetral, cifrado y auditoría



Auditoría cifrada en S3 con KMS.

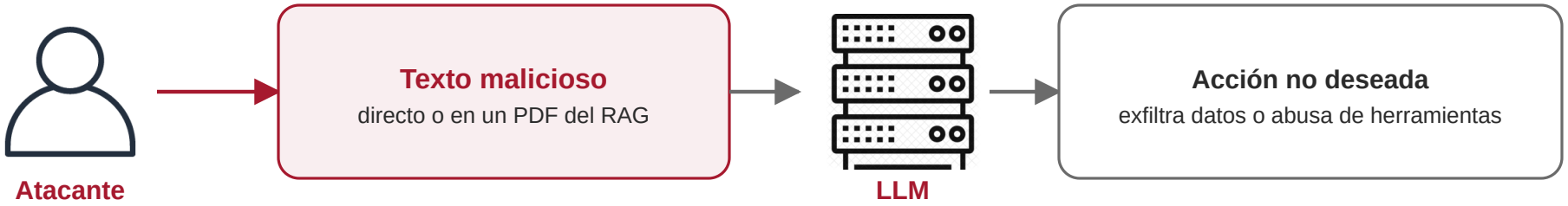
Capas: red, inspección, filtros y registro.

VPC = red virtual privada · TLS = seguridad de capa de transporte · WAF = cortafuegos de aplicaciones web · ALB = balanceador de carga de aplicación · KMS = servicio de gestión de claves · IA = inteligencia artificial

Fuente: AWS Security Reference Architecture (SRA); Machine Learning Security Guide

Defensas contra el riesgo número uno

Defensas arquitectónicas contra el riesgo número uno de OWASP



Objetivo: anular las reglas.

Defensas por capas, fuera del prompt:

1. Filtrar entrada y salida con Guardrails
2. Privilegio mínimo en herramientas (IAM)
3. Separar datos de instrucciones
4. Validar la acción antes de ejecutarla

PDF = formato de documento portátil · RAG = generación aumentada por recuperación · LLM = modelo de lenguaje de gran tamaño · IA = inteligencia artificial · OWASP = Proyecto Abierto de Seguridad de Aplicaciones Web

Fuente: OWASP Top 10 para aplicaciones con LLM 2025 (LLM01); AWS Bedrock Guardrails

SÍNTESIS INTEGRAL

Semana 14 en cuatro ideas fundamentales

Los cuatro fundamentos que conectan el modelo con la nube

1 · Paradigma

generación token a token

2 · Economía

tokens, ventana y Prompt Caching

3 · Servidores

Continuous Batching, TTFT y VRAM

4 · Decisión

gestionado o propio, con Ley 1581

Síntesis de la semana: del mecanismo del modelo a la decisión de despliegue.

TTFT = tiempo hasta el primer token · VRAM = memoria de la GPU

PRÓXIMA CLASE

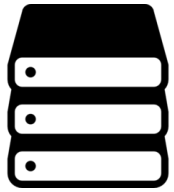
Adelanto de la Semana 15: GPU y cuantización

Del silicio a la compresión matemática de modelos



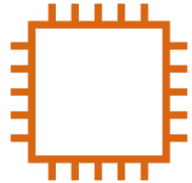
GPU

núcleos CUDA, Tensor
y memoria HBM



Cuantización

INT8, INT4 y AWQ:
menos bits, menos VRAM



Servicios cloud

despliegues en producción
y TCO real

Semana 15: optimización de modelos y aceleradores

GPU = unidad de procesamiento gráfico · CUDA = plataforma de cómputo paralelo para GPU · HBM = memoria de alto ancho de banda · INT8 = entero de 8 bits · INT4 = entero de 4 bits · AWQ = cuantización consciente de activaciones · VRAM = memoria de la GPU · TCO = costo total de propiedad

Conclusión y lecturas recomendadas

Documentación oficial, seguridad y marco regulatorio



AWS Bedrock Documentation

guías de usuario y arquitectura



NVIDIA Developer

documentación de aceleradores



OWASP Top 10 para LLM

ciberseguridad para aplicaciones LLM



Ley 1581 de 2012

protección de datos personales en Colombia

AWS = Amazon Web Services · NVIDIA = fabricante de GPU · OWASP = Proyecto Abierto de Seguridad de Aplicaciones Web · LLM = modelo de lenguaje de gran tamaño

Fuente: documentación de los proveedores citados en cada diapositiva